Assignment 1: Report

Martina Carminati, Sanna Plomp, Keval Mehta

November 7, 2026

1 Learning the Basics of PyTorch

1.1 Multi-layer Perceptron

In this first part, we started by setting a seed for reproducibility, and then importing the Fashion-MNIST dataset from `torchvision.datasets`. The training and test datasets were imported separately, and the training dataset was split into two subsets: what we will use for training (90%), and validation (10%). Setting batch size to 64, we created data loaders that will then load the data into the models.

To start off the systematic experimentation, we used a baseline model with the following parameters: `hidden_size` $=$ 256, `dropout` $=$ 0.2, `optimiser` $=$ `SGD`, and `learning_rate` $=$ 0.01. For validation, 10 epochs were used to get a good feel for the accuracy without overfitting. The experimentation is done sequentially with a small amount of values for each hyperparameter, more as a show of proof than to find the all-time highest possible accuracy.

In the first round, only the activation function was varied, while keeping the other parameters fixed. The following activation functions were used.

ReLU

$$f(x) = \begin{cases} x & x \geqslant 0 \\ 0 & \text{otherwise} \end{cases},$$

Leaky ReLU

$$f(x) = \begin{cases} x & x \geqslant 0 \\ mx & \text{otherwise} \end{cases},$$

GeLU

$$f(x) = x \cdot \Phi(x).$$

Leaky ReLU resulted in the highest validation accuracy of 0.8857. See Table 1 for the results of the different activation functions.

Activation	Optimiser	Validation Accuracy
ReLU	SGD	88.32%
Leaky ReLU		88.57%
GELU		87.73%

Table 1: Performance on validation dataset by different activation functions

Next, we will continue varying the optimisers, using ReLU as the activation function. All other hyperparameters were kept fixed, as SGD, Adam, and AdamW were tested. Adam resulted in the highest validation accuracy $=$ 0.8568. See Table 2 for the results of the different optimisers.

Continuing, we varied the learning rate. The following values were used for the learning rate [0.001, 0.01, 0.1]. Learning rate $=$ 0.001 resulted in the highest validation accuracy of 0.8872.

The size of the hidden layer was also varied. The following hidden sizes were tested: 128, 256, and 512. From the results, the hidden size of 256 was the best, with a validation accuracy of 0.8560.

Activation	Optimiser	Validation Accuracy
Leaky ReLU	SGD	84.77%
	Adam	86.58%
	AdamW	83.08%

Table 2: Performance on validation dataset by different optimisers

Finally, we tried out dropout values of 0.0, 0.2, and 0.5. Dropout = 0.0 resulted in the highest validation accuracy of 0.8925. The top 3 performing models were then retrained on the train set, and tested on the test set. The results are shown in Table 3. The best performing model was the one with Leaky ReLU, Adam, learning rate = 0.001, hidden size = 256, and dropout = 0.0, with a test accuracy of 0.8881.

Activation	Hidden Size	Dropout	Optimiser	Learning Rate	Test Accuracy
Leaky ReLU	256	0.0	Adam	0.001	88.81%
	256	0.2	Adam	0.001	88.29%
	256	0.2	SGD	0.01	83.24%

Table 3: Performance on Fashion-MNIST test set by different configurations

These top 3 models were then trained and tested on the CIFAR-10 dataset, adjusting them only for number of channels. Performance was not as strong, as demonstrated in Table 4. The best performing model was the one with Leaky ReLU, Adam, learning rate = 0.001, hidden size = 256, and dropout = 0.0, with a test accuracy of 0.4842.

Activation	Hidden Size	Dropout	Optimiser	Learning Rate	Test Accuracy
Leaky ReLU	256	0.0	Adam	0.001	48.42%
	256	0.2	Adam	0.001	43.54%
	256	0.2	SGD	0.01	44.02%

Table 4: Performance on CIFAR-10 test set by different configurations

The multi-layer perceptron model works by assigning weights to pixels, in comparison to convolutional structures, which takes into account neighbouring pixel values as well. This means that the network tries to learn weights by pixel, which is a strength for values that are invariate, applying roughly to the Fashion-MNIST dataset as images are centred and approximately at the same location, as compared to CIFAR-10, where the images' constituents move around within the images, and aren't always at the same location, which doesn't lead to a high accuracy.

Adding more layers and giving the network a capability to be more flexible would help increase accuracy, but the point remains that MLPs aren't the best tools for images.

Within the three results, a dropout of 0.2 does worse than no dropout, which may be because MLP needs all its capacity to perform, and dropout randomly sets some activations to 0 to prevent overfitting, which is not happening in this case, and hence dropout does not help performance.

1.2 Convolutional Neural Networks

Like in the previous section, a seed was set for reproducibility, and the dataset was imported. After importing the train and test separately, we split the training dataset into two subsets: what we will use for training (90%), and validation (10%). Setting batch size to 64, we created data loaders that will then load the data into the models.

We set up a basic convolutional neural network, similar to the one depicted in Figure 1, with two convolutional layers and max pooling between, with a customisable activation function.

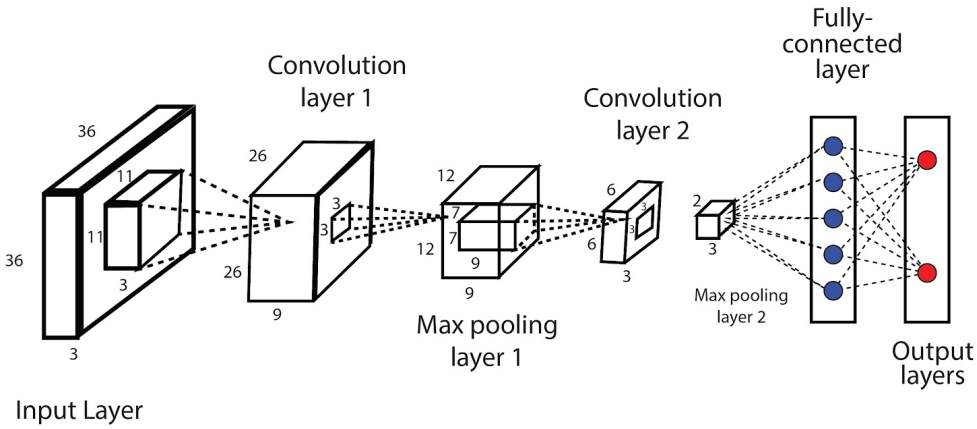


Figure 1: Sample Convolutional Neural Network

Defining functions for training, validating, and testing, we then moved to the systematic testing of parameters.
We performed this testing with 10 epochs for training, which was validated through the separated dataset. The test performance of that model was then calculated. The performance of the different activation functions is given in Table 5. As we can see in the table, GELU performed best, so we will use it to fit the rest.

Activation	**Optimiser**	**Validation Accuracy**
ReLU	SGD	81.25%
Leaky ReLU		81.12%
GELU		81.75%

Table 5: Performance on validation dataset by different activation functions

Activation	**Optimiser**	**Validation Accuracy**
GELU	SGD	81.75%
	Adam	90.13%
	AdamW	90.00%

Table 6: Performance on validation dataset by different optimisers

From Table 6, we can see Adam does best, so we will use that in our further analysis. Now, we test different parameter combinations of the optimiser we have chosen, and the results are present in 7. We can see that the best performing model, as per our combined analysis, is with the GELU activation function, Adam optimiser, learning rate$= 10^{-3}$, weight decay$= 10^{-5}$, and betas$= (0.9, 0.999)$. We see a consistently strong performance with these values of learning rate and betas, even when weight decay is varied, even though that does decrease it marginally.

Activation	Optimiser	Validation Accuracy	Learning Rate	Weight Decay	Betas
GELU	Adam	0.9035	10^{-3}	10^{-5}	$(0.9, 0.999)$
		0.9002	10^{-3}	10^{-5}	$(0.95, 0.995)$
		0.8960	10^{-3}	10^{-5}	$(0.99, 0.99)$
		0.9025	10^{-3}	10^{-4}	$(0.9, 0.999)$
		0.8948	10^{-3}	10^{-4}	$(0.95, 0.995)$
		0.8888	10^{-3}	10^{-4}	$(0.99, 0.99)$
		0.8837	10^{-2}	10^{-5}	$(0.9, 0.999)$
		0.8787	10^{-2}	10^{-5}	$(0.95, 0.995)$
		0.8878	10^{-2}	10^{-5}	$(0.99, 0.99)$
		0.8800	10^{-2}	10^{-4}	$(0.9, 0.999)$
		0.8623	10^{-2}	10^{-4}	$(0.95, 0.995)$
		0.8803	10^{-2}	10^{-4}	$(0.99, 0.99)$

Table 7: Performance on validation dataset by different parameters

Further, after this, we ran the models on the CIFAR-10 dataset, adjusting them only for number of channels. Performance was not as strong, as demonstrated in Table 8. After running the three best models from Table 7, we tested different optimisers as well, to see more examples, as shown in Table 9. None of these performed particularly well.

Learning Rate	Weight Decay	Betas	Test Accuracy
10^{-3}	10^{-5}	$(0.9, 0.999)$	0.6216
10^{-3}	10^{-4}	$(0.9, 0.999)$	0.6172
10^{-3}	10^{-5}	$(0.95, 0.995)$	0.6407

Table 8: CIFAR-10 performance on the best three parameter combinations of `optim.Adam`

Name	Test Accuracy
Adam	0.6288
AdamW	0.6289
SGD	0.4495

Table 9: CIFAR-10 performance on different optimisers

This could be due to the fact that in essence, Fashion-MNIST is a much simpler task, where the items of clothing are centred, and single-channel, and there is possibly a high level of separation between classes. The images of CIFAR-10 are not only RGB, but more complicated, and not as well separated. SGD performs much worse than Adam, which could be due to a weakly-trained learning rate on CIFAR, as opposed to Adam, which generalises better and isn't as dependent on the learning rate, as it uses the moments to adapt. While this difference is large, there is not much between Adam and AdamW, which may be due to both not overfitting, hence the decay is not a huge advantage.

Within the Adam optimiser, lowering the decay enables it to adapt faster, which may make it perform better on a complex dataset. I was not able to vary the architecture as much as I would've liked to, but adding more convolutional kernels may help the models to generalise better to the more complicated CIFAR-10 dataset.

2 Develop a "Tell-the-Time" Network

For the second task, we had to design, train, and evaluate a convolutional neural network on images of clocks to tell the time using the learnings from Task 1.

The "Tell-The-Time" is an image recognition task. The aim is to train a CNN that predicts the time from images of analog clocks as accurately as possible. To do so, three possible approaches are used: classification (treat the time as n-minute bins e.g., 30 minute); regression (treat the time as continuous e.g., mapping "03:00" $\rightarrow$ 3.0 and "05:30" $\rightarrow$ 5.5); and multi-head (time is binned in 60 classes for minutes and 12 classes for hours, and we produce two distinct outputs). The target is to find which method gives the lowest "common-sense" error: the absolute time difference between predicted time and true time.

2.1 Data and preprocessing

We are provided with a collection of 18,000 grayscale images of analog clocks with two possible resolutions: the original resolution (150$\times$150) and a down-sampled version (75$\times$75). To build the architecture of the models, the down-sampled version has been used, while the final results refer to the original resolution. We were provided also with 18,000 labels of size 2 (18000, 2) that represent the corresponding hours, and minutes (it's a supervised task).

After inspecting the shape of our data, we decided to convert the labels (e.g., 11:59) into the total number of minutes (e.g., 719) using the formula (hour $\cdot$ 60 $+$ minute), obtaining values from 0 to 719. To make the training easier, we standardised the pixels (originally from 0 to 255 into [0, 1]), using the mean and standard deviation of the whole dataset. Then, we performed a train-validate-test split just like in the previous tasks, using 80% training, 10% validation, and 10% test with `random_split` and shuffling to remove the influence of the order. To ensure reproducibility, we created the function `reset_seed()` that can be conveniently called whenever needed. Lastly, since the images were in greyscale, one channel dimension has been added to the image tensors.

Looking at the images and summary statistics of the dimensions of those images, two aspects arise:

- The images are slightly tilted (in 3D), rotated, zoomed-in, and contain small bright spots. One possible way to handle this problem is by using data augmentation: by randomly transforming the original image, the model can learn to be more robust to such differences. To do so, we can use `torchvision.transforms.v2` from PyTorch.

- The circular nature of time: because time is cyclical, at the boundary 12:00 - 00:00 two predictions such as 11:59 and 00:02 are only 3 minutes apart, but when we convert them into total minutes (11:59 = 11*60 + 59 = 719 minutes and 00:02 = 0*60 + 2 = 2 minutes), and we take the default error, they seem to be 719 - 2 = 717 minutes apart. One possible way to account for this is to take the minimum distance on both sides of the clock, so that the difference never exceeds half of the clock.

To be able to compare the results from the three sub-tasks, classification, regression, and multi-head, we decided to build a common CNN baseline. Only the output layers for each task's CNN and the loss function for the regression task are different. We selected the best baseline setting for the CNN using the 75 $\times$ 75 images dataset, with the classification approach using 30-minute bins (24 classes). We built the very first CNN option as similar as possible to the CNN from Task 1, then we tried to improve from there.

All the models were compared using "common-sense error" on the validation set:

$$e(t, \hat{t}) = \min\left(|t - \hat{t}|,\ 720 - |t - \hat{t}|\right),$$

where t is the true time, and $\hat{t}$ represents its prediction, both in minutes, averaged over all images. Using the term $720 - |t - \hat{t}|$ allows to account for the problem that arises at the boundary.

Since we are using the classification approach, the predicted classes were then re-converted into minutes by taking the midpoint of the bin. Experiments 1, 2 and 5 were trained using 20 epochs and 3 different seeds, and then we compared the results, after we computed the mean and standard deviation of the resulting error. In this way, we can make sure that the results are not due to random chance. Experiment 3 was run on one single seed, because in early runs we noticed that the differences between the learning rates were large enough (so we didn't need to pool the results), and using 40 epochs, because a small learning rate (e.g., lr = 0.0003) needs more time to train. Lastly, experiment 4 was run using 50 epochs, because its point was to check when the curves stopped learning (no multiple seed was needed).

Depth As a baseline, we used the CNN of Task 1; we added padding to each convolutional layer, so that the 3×3 convolutions kept the size of the images and only the pooling shrunk it, and we removed one fully connected layer to reduce the number of parameters. Then we built two more CNNs, adding one convolutional layer at a time. Each layer is a 3×3 convolution, ReLU, and 2×2 max pooling. Results are reported in Table 10. We can notice that adding convolutional layers helps the model inspect the image. Each pooling halves the image, so the deeper layers look at larger parts of the clock. The network with four convolutional layers reported the lowest error, therefore we selected it.

Depth	Mean	Std
2	45.828	1.705
3	23.483	1.904
4	19.498	2.820

Table 10: Pooled validation results for CNNs depths.

Weights initialisation Different weight initializations can influence the way our network learns. Therefore, we compared the PyTorch default initialization, with He (that is suited for ReLu) and Xavier (that was suggested in one of the exercises of the book "Dive into Deep Learning"), with biases set to zero. The results are displayed in Table **??**, and as we can see there was almost no difference between the three initializations, therefore we kept the default setting.

Learning rate Then we experimented with learning rates. We tried three different values: $3 \cdot 10^{-3}$, 10^{-3}, and $3 \cdot 10^{-4}$. We used only one seed (because the differences were large enough) and 40 epochs (because smaller learning rates learn more slowly). The learning rate of 10^{-3} performed best, with an error of 14.44 minutes, therefore is selected.

Number of epochs To check how many epochs were needed, we tried training our selected CNN for 50 epochs, and tracked the training loss, validation loss, and validation error (Figure 2). In the first three epochs the loss was stuck at approximately 3.18 with an error of 176.41. The value 3.18 is approximately the logarithm of the number of classes (24): this means that the model is still guessing at random. After that initial plateau the error dropped quickly, reaching an error of 18.03 at around 30 epochs, and 16.94 at 40 epochs. However, looking at Figure 2, we can see that after 30 epochs (more or less) the training loss keeps going down while the validation loss starts to rise. This might be a sign of overfitting. Based on this, we kept the number of epochs low (20 epochs) to train the models on the 75×75 images, since it's an easier task, and we increased it to 40 epochs for the 150×150 dataset.

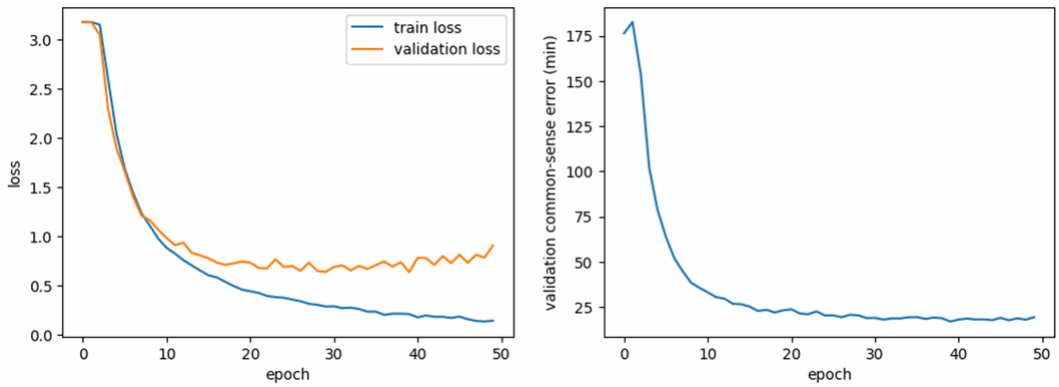


Figure 2: Training and validation loss (left) and validation common sense error (right) of the four-block CNN over 50 epochs

Batch size Lastly, we compared three possible batch sizes: 32, 64, and 128. We run this experiment using 20 epochs and 3 seeds and we averaged the results. A batch size of 64 was our baseline (used for Task 1), batch size 32 performed slightly better, while batch size 128 gave unsatisfactory results. The improvement obtained by using batch 32 was very small, and since the training is slower, so we kept 64.

Final network The final CNN used in all the three sub-tasks of Task 2 is shown in Table 11. Only the output layer changes between the classification, regression, and multi-head models, and for the regression task we had to use a different loss function.

Layer	Output size
Input	$1 \times 75 \times 75$
Conv 3×3 (6), ReLU, MaxPool	$6 \times 37 \times 37$
Conv 3×3 (16), ReLU, MaxPool	$16 \times 18 \times 18$
Conv 3×3 (32), ReLU, MaxPool	$32 \times 9 \times 9$
Conv 3×3 (64), ReLU, MaxPool	$64 \times 4 \times 4$
Flatten, Linear, ReLU	120
Linear (output)	depends on the task

Table 11: Base CNN used for Task 2

Additionally, all the three sub-tasks (Classification, Regression, and Multi Head) used the same number of epochs (20), Adam optimizer with learning rate of 0.001, the same seed, the same train-validation-test split and the same error metric.

2.2 Classification

The CNN for the classification task, was the selected CNN presented in Table 11, with the following output layer `self.fc2 = nn.Linear(120, num_classes)`, a fully connected layer with as many outputs as the number of classes.

We created the train and validate functions, and to compute the common-sense error, we re-converted the predicted classes into minutes by taking the midpoint of the bin.

For the classification task time was divided into bins of equal size, and each bin is a class. We run an experiment to test the performance of the network with different bin sizes of 30, 15, 5, and 1 minutes.

The 30-minute bins produced the lowest common-sense error, with a test error of 17.18 minutes. Using the other bin sizes, the error grows quickly as the bins get smaller. With 1-minute bins the training loss stayed at about 6.58 (which is the logarithm of the number of classes$=\ln 720$) for all epochs, so this means that the model never learned, it was simply random guessing. For this reason, we run a second experiment, training for 100 epochs the model with 1-minute bins. However, the results did not change, which means that the network is not powerful enough for this task.

This shows the limitations of high-cardinality classification:

- With more classes, the task is harder to fit, because here 1-minute classes are harder to distinguish. The model has fewer example images to use to learn.
- The loss function for the classification task does not account for how "off" the predictions are: all wrong predictions are simply classified as "incorrect". The model does not distinguish between a prediction that is one minute off and one that is hours away from the true value.
- Dividing a continuous variable into bins always causes loss of information. In addition, we used the midpoint to predict the bins, so even a perfect classifier would have an error.

2.3 Regression

For the regression task, time is treated as one continuous number, converted into hours using hour $+$ minute$/60$ (so 3:30 becomes 3.5).

The CNN is the same as before, with the difference that the output layer has a single neuron. We compared three loss functions with 20 epochs.

Here we run an experiment to check which loss function would be the best. We tried out three different loss functions: MSE Loss, MAE Loss and SmoothL1 Loss. The MSE loss penalises strongly the magnitude of the errors by taking their squared value, therefore large errors get a very high penalty. This means that if there is a very bad picture, that gives a huge error, it will severely affect the training process; the MSE is very sensitive to outliers. MAE, on the other hand, penalises every unit of error equally. For this reason it is less sensitive to outliers, but this also means that small errors are penalised just as much as the large ones. The Smooth Loss is a mix of the two: for errors smaller than the threshold it behaves like the MSE (so the gradients get shrunk), but for errors larger, it behaves like MAE (penalising units equally).

We found out that `SmoothL1Loss` performed best, with a test error of 65.20 minutes. This error is still very high and is much worse than classification. The main reason why this happens is the problem at the boundary at 11:59 - 00:00 mentioned above. Two predictions such as 11:59 and 00:02 are only 3 minutes apart, but as regression predictions they are respectively 11.98 and 0.03 hours, almost 12 hours apart. This is due to the circular nature of time, but it causes a big problem: the loss function doesn't know that these two predictions are actually close, therefore it penalises them as if they were actually 12 hours apart. We handled this problem only in the validation function by taking `torch.remainder(outputs, 12)` and `torch.minimum(difference, 720 - difference)` to compute the "common sense error". However, this aspect is not handled in the training.

2.4 Multi-Head Models

For the multi-head task, we kept as usual the baseline CNN, and then we split the output layer into two heads: one with 12 outputs (for the hours) and one with 60 outputs (for the minutes). The forward pass returns both outputs. Both heads are trained with cross-entropy, and the two losses are combined as

$$L = L_{\text{hour}} + \alpha \cdot L_{\text{minute}}.$$

The predicted time is $\hat{h} \cdot 60 + \hat{m}$, where $\hat{h}$ and $\hat{m}$ are the classes with the highest score in each head. We tested $\alpha \in \{0.5, 1, 2\}$ with 20 epochs.

We then tested different alpha values to combine the losses from the two heads, and found out that $\alpha = 0.5$ performed best, with a test error of 12.96 minutes, the best result so far on the 75×75 images. A possible explanation is the fact that a small α gives more weight to the loss on the hour. This makes sense for the common-sense error: a wrong hour is off by at least 60 minutes. The model with $\alpha = 1$ did not learn at all: its loss stayed at ≈ 6.58. The model with $\alpha = 2$ was also stuck for the first epochs and then started learning. The results are shown in Table 12.

α	**Validation Error (min)**
0.5	13.86
1.0	181.48
2.0	29.93

Table 12: Multi-head model with different values of α on the 75×75 images (20 epochs)

The multi-head model works better than both the classification task and the regression task. The reason why is that it follows the same structure of the classification task (which already performs better than regression), and then splits the classification problem in two smaller parts. Instead of 720 classes, it has 12 and 60 classes.

2.5 Comparison and 150×150 Images

Finally, we trained all the three approaches again on the 150×150 images dataset, using the best settings from the 75×75 experiments and 40 epochs for all of them. Here, we also added the 1-minute classifier, because it allows for a more fair comparison between the approaches: classifying on 30-minutes bins is clearly easier than both the regression and multi-head task. The comparison is fair only if we compare them while using

Formulation	Setting	Test Error 75×75 (min)	Test Error 150×150 (min)
Classification	30-min bins	17.18	23.55
Classification	1-min bins	–	130.40
Regression	`SmoothL1Loss`	65.20	49.25
Multi-head	$\alpha = 0.5$	12.96	176.73

Table 13: Test common-sense error of the best models. Comparison between 75x75 and 150x150 resolutions results

1-minute bins for classification, in this way we have the same resolution as the regression and multi-head models. The results are in Table 13.

On the 75×75 images, the multi-head model performed best (12.96 minutes), followed by 30-minute classification (17.18 minutes) and regression (65.20 minutes). On the 150×150 images, the results are completely different. The multi-head model did not learn at all: its loss was stuck at the random-guessing value for all 40 epochs, and its error (176.73 minutes) is extremely high and not acceptable. Classification with 30-minute bins became the best model, with 23.55 minutes, although it's also the easiest task to perfom, and the result is still worse than the one obtained the small images. The results of the regression approach improved, from 65.20 to 49.25 minutes. A possible explanation is the fact that the regression task might have benefitted from training for 40 instead of 20 epochs. Lastly, the 1-minute classifier now learned, but it has also significantly overfitted: its training loss went down to 0.41, while its test error stayed at 130.40 minutes.

The bigger images did not help our models: the network was chosen and tuned on the 75×75 images and it shows. Possible solutions to improve the results on the 150×150 images could have been: data augmentation (with larger resolution the model has finally overfit); a deeper network (a fifth convolutional layer could have produced a feature map of a size similar to the one we tuned the parameters for).

2.6 Limitations

- A problem we saw in many experiments is that some models never start learning: the loss was stuck at the random-guessing value and the common-sense error was very high (e.g., 180 minutes). This happened with: learning rate of 0.003, with batch size 32, with $\alpha = 1$, and with the multi-head model on the 150×150 images. The other experiments were stuck for a few epochs and then started learning.
- The model we tuned the parameters on, the one for the 75×75, was underfitting: looking at the images, we were supposed to apply data augmentation to account for the random transformation that the images presented. However, data augmentation could have helped if the model was overfitting, but here it was doing the opposite. Therefore, no data augmentation was applied. It might have improved the results on the 150×150 images.
- We used the same CNN for different tasks. There are two main reasons why this was the chosen approach: (1) computational limitations, an external GPU was used, but this task is still computationally heavy; (2) comparability, by using the same structure, we were able to have a more fair comparison of the models. However, a separate definition might have improved the results.

3 Contributions

Martina Task 2 code, task 2 report

Sanna Task 1 code, task 1 report

Keval Task 1 code, task 1 report